

Expert Report: Elicitation Resources and Techniques in Software Requirements Engineering

Section 1: Foundations of Requirements Elicitation (REI)

1.1 Defining Requirements Elicitation: Active Discovery

Requirements Elicitation (REI) is the foundational process within Software Requirements Engineering (SRE) that focuses on identifying, gathering, and profoundly understanding the needs, desires, and constraints articulated by stakeholders for a new or modified software system. It is defined as a systematic and disciplined approach to specification and management, with the goal of minimizing the risk of delivering a system that fails to meet stakeholder expectations.

The specialized term "elicitation" is employed deliberately in SRE rather than "gathering" or "collection". This distinction emphasizes that good requirements cannot simply be collected passively from a client by asking surface-level questions; they must be actively *drawn out* and discovered by the analyst through deep investigation. The requirements analyst's role is therefore diagnostic, requiring persistence in encouraging stakeholders to explain the rationale behind their proposed solutions, thereby exposing the underlying necessity.

Effective elicitation seeks to capture four critical layers of needs:

1. **Explicit Needs:** Requirements that are stated directly by the stakeholder (the obvious requests).
2. **Implicit Needs:** Requirements that are generally understood within the operational domain but are often not formally stated (e.g., standard performance or availability).
3. **Tacit Needs:** Unspoken knowledge derived from deeply habitual workflows or personal experience that stakeholders assume are "obvious" to everyone involved. Uncovering tacit knowledge is often the hardest challenge in REI.
4. **Future Needs:** Anticipated needs, strategic objectives, or long-term operational goals that might not be immediately necessary but influence system architecture and design.

1.2 Objectives and Value Proposition in the SDLC

Requirements elicitation acts as the critical initial step in the overall Requirements Engineering (RE) process, serving as the cornerstone for all subsequent stages of software development. The primary objectives of effective REI are threefold: to identify the business need, scope, assumptions, and risks; to achieve consensus among diverse stakeholders regarding these needs; and ultimately, to create a comprehensive and accurate requirements document. This clarity ensures the software development team fully understands exactly what must be built to meet expectations.

The value proposition of rigorous elicitation is significant, directly impacting project success and

cost efficiency:

- **Risk Mitigation:** Proper elicitation supports the identification of potential risks, security vulnerabilities, and problems early in the development cycle, reducing the likelihood of project failure due to unmet requirements.
- **Cost Efficiency:** Understanding requirements upfront minimizes the chance of costly reworks and delays that inevitably stem from incomplete or misunderstood requirements discovered later in the Software Development Life Cycle (SDLC).
- **Quality Enhancement:** Detailed, well-understood requirements contribute to better design, implementation, and testing processes, resulting in a higher quality final product.
- **Clarity and Expectation Management:** The process clarifies customer expectations, avoids misunderstandings, and manages the inevitable tensions that arise when satisfying multiple stakeholders.

1.3 The Foundational Elicitation Process Cycle

Requirements elicitation is inherently an iterative process that involves continuously refining and updating requirements based on feedback from stakeholders. This iterative nature serves as an adaptive strategy, functioning as continuous risk mitigation against the primary challenge of requirement volatility—the tendency of needs to change over time. A purely linear approach would carry a prohibitively high risk of failure in modern software development contexts.

The cycle typically progresses through the following steps :

1. **Preparation and Planning:** Identifying stakeholders, defining project scope and objectives, selecting appropriate techniques, and preparing resources.
2. **Elicitation Application:** Executing chosen techniques, such as interviews or observation, to discover and capture needs.
3. **Requirements Documentation:** Structuring the gathered information into formal documents like Use Cases or User Stories.
4. **Analysis and Negotiation:** Analyzing the documented requirements to identify conflicts, redundancies, and gaps, followed by negotiation to prioritize requirements and manage trade-offs.
5. **Validation and Verification:** Confirming the requirements are complete, consistent, acceptable to all stakeholders, and align with core business objectives.

Section 2: Elicitation Resources and Stakeholder Management

Effective requirements discovery relies on tapping into the correct resources and establishing effective communication protocols with all involved parties.

2.1 Primary Elicitation Resources

The elicitation process draws information from three core reservoirs :

1. **People (Stakeholders):** Individuals or groups who have vested interests in the system's outcome, providing qualitative data, opinions, and aspirational goals. This is the source of critical tacit knowledge.
2. **Documents:** Existing organizational artifacts, formal specifications, and contracts that provide crucial historical context, operational rules, and formal constraints.

3. **Existing Systems and Environment:** The current operational landscape, including physical infrastructure, workflows, technological constraints, and existing interfaces with other systems.

2.2 Comprehensive Stakeholder Classification and Roles

Stakeholders are individuals or groups who can be impacted by or have an impact on the project's success. Successful elicitation requires segmentation and strategic engagement based on their roles:

- **End-users:** Directly interact with the system; they provide essential functional, usability, and workflow requirements. They are the primary source for identifying efficiency pain points.
- **Customers/Clients (Sponsors):** Individuals or organizations commissioning and funding the project. They define the highest-level business objectives and key performance indicators.
- **Business Managers/Owners:** Define the strategic alignment of the system, necessary operational changes, and prioritization of features based on organizational value.
- **Technical Experts/Development Team:** These include software engineers and architects who assess technical feasibility, constraints (e.g., security, scalability), and system-to-system integration needs.
- **Regulatory Bodies/Auditors:** Define non-functional requirements such as legal compliance, data governance standards, and specific mandated security protocols.

A challenge in managing diverse stakeholders is the risk of technical bias, where requirements may be weighted towards architectural soundness (prioritized by technical experts) at the expense of utility or user experience (prioritized by end-users). The requirements analyst must consciously balance *feasibility* (technical constraints) with *utility* (user adoption) and *value* (business owners) throughout the prioritization and negotiation phases.

2.3 Stakeholder Engagement Strategy: Dos and Don'ts

Effective engagement must follow structured protocols to ensure comprehensive requirement capture:

Dos: Best Practices	Don'ts: Avoidances
Engage stakeholders early and frequently.	Rely solely on one technique; use a mix for robust insights.
Keep communication channels open and transparent.	Assume stakeholder needs without explicit validation.
Use multiple elicitation techniques for comprehensive coverage.	Permit the backlog of "To Be Determined" (TBD) requirements to grow uncontrolled.
Document requirements clearly using structured templates.	Fail to acquire adequate domain expertise.
Validate and prioritize requirements proactively (e.g., using mockups).	Allow confusion or misunderstandings to persist.

Section 3: Taxonomy of Elicitation Techniques

The selection of elicitation techniques must be intentional, based on the project's scale, the inherent risks, and, most importantly, the specific type of knowledge being sought (stated vs.

tacit). No single method can provide a complete set of requirements; therefore, combining techniques is a mandatory strategy for robust SRE.

3.1 The Expert Classification Framework (Types of Interaction)

Elicitation methods are classified primarily by the nature of the interaction they facilitate:

Category	Interaction Type	Primary Goal	Example Techniques
Conversational (Direct)	High, Scheduled Contact	Collecting stated needs, opinions, goals, feelings	Interviews, Brainstorming, Workshops (JAD), Questionnaires
Analytic (Indirect)	Low/Passive Contact	Understanding existing processes, rules, system boundaries	Document Analysis, Interface Analysis, Existing System Study
Observational (Indirect)	High, Contextual Contact	Uncovering tacit knowledge, real behavior, pain points	Ethnography, Shadowing, Contextual Inquiry
Synthetic (Model-Driven)	Mixed Contact	Validating conceptual understanding, clarifying ambiguous requirements	Prototyping, Use Cases, User Stories/Scenarios

3.2 The Trade-off Paradigm in Technique Selection

The requirements analyst must navigate fundamental trade-offs when selecting techniques:

- **Breadth vs. Depth:** Techniques such as large-scale surveys offer high **breadth**, gathering data from many individuals, but they lack the nuance and context required for deep understanding. Conversely, observational methods like ethnography sacrifice breadth but provide highly contextualized data necessary for sophisticated understanding of causal mechanisms and subjective viewpoints.
- **Cost vs. Time:** Certain techniques, particularly observational methods and deep, one-on-one interviews, are inherently time-consuming and costly. Observational methods are notably inefficient when the project operates under a very tight requirement schedule. This means that the time constraint itself acts as a non-functional requirement that heavily dictates the appropriate process choice. Projects with severe time limitations must consciously lean towards faster, broader methods (Conversational/Analytic) and use model-driven techniques for high-risk areas, accepting a greater risk of missing subtle, deep tacit knowledge.

Section 4: Detailed Review of Direct and Analytical Techniques

4.1 Conversational Methods (Direct Interaction)

4.1.1 Interviews

Interviews remain the most common and often most effective method for knowledge transfer

between the customer and the requirements analyst. They are one-on-one sessions designed to gather detailed insights into customer expectations, opinions, and goals.

- **Strengths:** Interviews facilitate the collection of rich information, allowing the analyst to uncover opinions, feelings, and the underlying goals of different individuals. They allow for direct, open-ended questions that elicit honest, unbiased, and complete responses.
- **Ambiguity Management:** While ambiguity in communication is often seen as an obstacle, it can be strategically managed. Ambiguity in requirements, particularly in the initial interview stages, often signals that a requirement is still a "tacit idea" that needs to be surfaced. The analyst must leverage this ambiguity to prompt the stakeholder for elaboration and context, ensuring correct disambiguation and deeper knowledge extraction.
- **Weaknesses:** Interviews are time-consuming and costly, especially when dealing with multiple stakeholders. They are highly dependent on the stakeholder's verbalized thought and accurate recall of experiences.

4.1.2 Brainstorming and Workshops (JAD)

These methods bring stakeholders together to discuss project objectives and rapidly generate a broad set of ideas and potential needs. Joint Application Development (JAD) workshops are highly structured, facilitator-driven processes that aim to collect requirements collaboratively.

- **Strengths:** They rapidly elicit many requirement ideas and encourage diversity of thought and creative input through free-form discussion. Facilitator-driven processes, such as JAD, can be more efficient in gathering requirements than conventional, sequential interview methods.
- **Weaknesses:** They require highly skilled facilitation to manage group dynamics and prevent political conflict. The resulting broad set of ideas must be diligently distilled by the Business Analyst into a defined set of actionable needs.

4.1.3 Surveys and Questionnaires

These are conversational techniques administered indirectly to gather uniform information from a large, potentially dispersed stakeholder group. While they do not provide the depth of an interview, a well-designed questionnaire can significantly improve the efficiency of subsequent stakeholder meetings by clarifying baseline assumptions and eliminating unnecessary follow-up sessions.

4.2 Analytical Methods (Indirect Investigation)

These methods focus on objective, written information, providing a crucial historical and contextual baseline.

4.2.1 Document Analysis

Document analysis involves reviewing existing published material to gather details about current processes, formal rules, contracts, and system behavior. This method is typically performed early in the elicitation process to establish initial context.

- **Strengths:** Document analysis is a key organizational resilience mechanism; it leverages existing material, meaning the process does not start from scratch, and it is particularly

essential when Subject Matter Experts (SMEs) are unavailable or have left the organization. It can provide a crucial means to cross-check information gathered through other, more subjective methods.

- **Analyzed Materials:** Contracts, policy manuals, training manuals, business plans, Requests for Proposal (RFPs), and existing system files.
- **Weaknesses:** The method is limited exclusively to 'as is' information, offering no inherent insight into future needs or desired process improvements. Existing documentation may also be outdated, incomplete, or irrelevant to the planned system changes. Therefore, relying solely on analytical methods for a project involving process redesign risks automating existing inefficiencies, making cross-validation with dynamic, conversational data mandatory.

4.2.2 Interface Analysis

Interface analysis is the process of observing and analyzing how a system interacts with external entities, including users, other systems, and hardware.

- **Focus Areas:** Identifying user interfaces, system-to-system interfaces, and critical data flows. This analysis determines who uses each interface, the tasks performed, and the required features needed to support identified use cases.
- **Strengths:** This technique provides valuable, objective detail about systemic requirements, inputs, outputs, and dependencies. By identifying these integration requirements early, the process saves time and expense in planning accurate testing and integration phases.
- **Weaknesses:** Interface analysis is inherently limited in capturing stakeholder perspective, subjective desires, or non-system business process requirements that define the *why* of the interaction.

Section 5: Model-Driven and Observational Techniques

5.1 Synthetic/Model-Driven Techniques

Synthetic methods focus on developing models, diagrams, or working versions of the system to clarify features and validate conceptual understanding.

5.1.1 Prototyping

Prototypes are simulated shells of the final application used primarily to gather requirements related to the User Interface (UI), look, feel, and flow.

- **Throwaway vs. Evolutionary Prototypes:**
 - **Throwaway Prototype:** Built rapidly to quickly uncover and clarify ambiguous system interface requirements and is intended to be discarded after the necessary requirements are confirmed. This approach signifies that the initial requirement clarity is low, necessitating experimentation before committing to a final design.
 - **Evolutionary Prototype (Breadboard):** This differs fundamentally, as the goal is to build a robust prototype that forms the heart of the new system, which is then constantly refined and built upon. Choosing this method implies higher confidence

in the foundational requirements stability, justifying the early investment in robust code.

- **Functional Scope Categories:**
 - **Horizontal Prototype:** Models a shallow and wide view of the system's functionality, focusing primarily on the visualization layer without significant underlying business logic.
 - **Vertical Prototype:** Models a deep and narrow slice of the system's functionality, often including some built-in business logic to test and uncover complex requirements in a specific area.
- **Integration Necessity:** Prototyping is not a standalone technique. It is essential that the UI requirements gathered via prototypes be integrated with established use cases, scenarios, data models, and business rules; failure to do so results in a costly mistake and an incomplete requirement set.

5.1.2 Use Cases, User Stories, and Scenarios

These are structural mechanisms used to document, organize, and communicate functional requirements, often in conjunction with prototyping.

- **User Stories (Agile Focus):** These are informal descriptions of a system function, typically written as: "As a [role], I want [goal], so that [reason/value]." They focus on the user need and the business goal, often used in agile projects to facilitate the emergence of detail organically.
- **Use Cases (Formal Focus):** Use cases are formal specifications (often part of UML) that detail the interaction between an actor and the system to realize a single functional requirement. They specify the intended functionality from the user's perspective, system behavior, and expected outcomes.
- **Complementary Application:** For maximum value, the analyst should start with a User Story to capture the user need and business goal. A detailed Use Case or scenario should then be developed around the story if the implementation is complex or carries high risk, ensuring the required level of functional detail is specified.

5.2 Observational Techniques (Uncovering Tacit Knowledge)

Observational methods are crucial for understanding the application domain and capturing knowledge that stakeholders cannot or do not articulate verbally (tacit knowledge).

5.2.1 Ethnography and Contextual Inquiry

Ethnography involves observing the target audience firsthand in their work environment to develop insights based on real behavior rather than self-reported information. Contextual inquiry is a related method focused on understanding the user in the context of their daily tasks.

- **Value Proposition:** This method is excellent for exploratory research because it provides unmediated access to the phenomena being studied, which bypasses the "measurement problems" inherent in interviews and surveys (such as faulty recall or discomfort in relaying difficult truths). It reveals pain points, gaps, and opportunities that users, due to deep familiarity, may not think to mention themselves. It yields highly nuanced and sophisticated understandings of workflow and subjective viewpoints.
- **Trade-off:** The strength of depth and rich context comes at the expense of breadth;

ethnography is time-intensive and cannot gather data about as many individuals or locales as a survey can.

5.2.2 Process Shadowing and Collaborative Observation

Shadowing involves monitoring ongoing operational processes and human activities to gather tacit, non-verbal communication and understanding of the domain. Collaborative observation extends this by involving an ongoing confirmation or deeper analysis across all phases of system development. This technique is vital when customers cannot verbally expose their needs and the analysis must derive requirements by watching how the system *is* currently being used.

Section 6: Comparative Analysis of Core Elicitation Techniques (Easy Notes Matrix)

Selecting the optimal technique requires a clear understanding of the trade-offs between depth, efficiency, cost, and the type of information sought. The table below synthesizes the utility of the five major technique categories.

Table 2: Comparative Matrix of Core Elicitation Techniques

Technique	Strengths (S)	Weaknesses (W)	Best Use Case
Interviews	Rich, detailed info; uncovers opinions/feelings; allows follow-up questions	Time-consuming; high ambiguity risk; dependent on stakeholder recall and verbalization	Early stage; gathering specific depth from key decision-makers; complex, non-visual requirements.
Brainstorming/Workshops	Elicits many ideas quickly; encourages diversity/free thought	Requires creative participants; ideas require BA distillation and refinement; high risk of political conflict	Defining scope boundary; exploring new solutions; high-level feature ideation; consensus building.
Document Analysis	Leverages existing material; cost-effective; cross-checks other methods	Limited to 'as is' information; documents may be outdated or incomplete; no insight into future state	Initial phase research; understanding existing system rules/structures when Subject Matter Experts (SMEs) are unavailable.
Ethnography/Observation	Uncovers tacit knowledge (real behavior); high face validity; identifies unexpected pain points	Time-consuming and costly; sacrifices breadth for depth; inefficient with tight schedules	Complex processes; systems with high user interaction; identifying true pain points in existing workflows/UX.
Prototyping	Clarifies UI/flow requirements quickly; facilitates hands-on feedback; excellent for	Not a standalone method; difficult to integrate UI requirements with deep	High-risk UI projects; validating efficient workflow; addressing ambiguous interface

Technique	Strengths (S)	Weaknesses (W)	Best Use Case
	interface requirements	business logic; can distract from core needs	requirements (look and feel).

Section 7: Challenges, Conflict Resolution, and Best Practices

7.1 Common Pitfalls in Elicitation

Despite the use of sophisticated techniques, requirements elicitation faces several persistent challenges that must be proactively managed:

- **Communication Gaps:** This is a primary challenge, arising from language barriers, the use of technical jargon, or differing interpretations between technical staff and non-technical business stakeholders.
- **Conflicting Priorities:** Stakeholders often possess different goals based on their roles (e.g., security for IT vs. usability for marketing), which can lead to competing requirements and project delays.
- **Unspoken Needs (Tacit Knowledge):** Stakeholders familiar with their workflows may fail to articulate needs they deem self-evident, leading to critical omissions in the requirements document.
- **Ambiguous or Incomplete Requirements:** Vague descriptions that lack sufficient detail (e.g., "The system must be secure") create confusion, rework, and missed deadlines.
- **Unmanaged Volatility:** Requirements may change over time, making it difficult to maintain documentation alignment and necessitating additional work to keep the project on track.

7.2 Conflict Resolution Framework

Conflicting requirements, which naturally arise from misaligned stakeholder goals, require a systematic, structured resolution framework :

1. **Identify and Document Conflicts:** The requirements analyst must detect competing requirements early in the elicitation process.
2. **Analyze Stakeholder Needs:** Understand the root cause of the conflict by drilling into the stakeholder rationale behind each requirement.
3. **Apply Prioritization Techniques:** Standardized prioritization methods (e.g., MoSCoW) must be used to collectively rank the competing requirements based on perceived business value.
4. **Negotiate and Agree on Trade-Offs:** Facilitate discussions to manage expectations and achieve consensus, often resulting in necessary compromises on scope or timeline.
5. **Validate Alignment with Business Objectives:** Ensure the negotiated solution and final scope remain fully aligned with the primary strategic goals of the organization.

A powerful tool in this process is the requirements traceability matrix, which functions as a formal arbitrator. By tracing conflicting requirements back to the foundational, agreed-upon business objectives, the analyst can objectively demonstrate which requirement provides higher strategic value, thereby depersonalizing the conflict and objectively facilitating necessary

trade-offs.

7.3 Mitigation Strategies and Professional Best Practices

Challenge	Root Cause	Mitigation Strategy (Actionable)
Communication Gaps	Jargon use, differing interpretations	Use a facilitator; employ simple, clear language; utilize visual aids (flowcharts, diagrams).
Conflicting Priorities	Misaligned goals among diverse stakeholders	Host structured prioritization workshops; use traceability matrices to align requirements with core business goals.
Unspoken Needs/Tacit Knowledge	Stakeholders assume knowledge is "obvious"	Conduct Contextual Inquiries/Ethnography; use persistent probing "why" questions during interviews.
Ambiguous Requirements	Vague descriptions without sufficient detail	Use models (Prototypes, Use Cases) to confirm understanding; enforce structured documentation standards.
Unmanaged Changes (Volatility)	Lack of control over scope and documentation update	Establish a formal change control process; centralize documentation in version-controlled tools; regularly review and validate against the baseline.

A central theme in successful mitigation is the shift toward **Visual Aids as the Essential Communication Protocol**. Given the inherent ambiguity of natural language and the diverse professional backgrounds of stakeholders, visual models such as prototypes, flowcharts, and wireframes are necessary to bridge communication gaps. These visuals transition abstract, linguistic concepts into concrete, shared mental models, significantly enhancing the reliability of requirement confirmation and minimizing the risk of misinterpretation. Furthermore, establishing a standardized documentation process and centralizing requirements in structured tools (like JIRA or Confluence) is essential for easy access and version control.

Conclusion

Requirements elicitation is recognized as the single most critical, yet non-trivial, phase in Requirements Engineering. It is defined not by the passive collection of data but by the active, diagnostic discovery of explicit, implicit, and tacit needs from diverse resources. The complexity of modern software systems mandates the abandonment of reliance on single techniques; a robust process requires combining conversational methods (Interviews, Workshops) for rationale and detail, analytical methods (Document Analysis, Interface Analysis) for context and constraint definition, and observational methods (Ethnography) for surfacing crucial tacit knowledge and validating real-world pain points. The process must be iterative, using feedback

loops (e.g., validation via prototypes) as a continuous risk mitigation strategy against volatility. Ultimate success is determined by the analyst's ability to manage stakeholder conflicts objectively through prioritization techniques and traceability, and by replacing potentially ambiguous verbal communication with shared, validated visual models.

Works cited

1. Requirements Elicitation in Software Engineering: A Complete Guide, <https://www.testbytes.net/blog/requirements-elicitation/>
2. Requirements Elicitation: The First Step of Product Requirements Engineering - Product School, <https://productschool.com/blog/skills/requirements-elicitation-product-requirements-engineering>
3. (PDF) Requirements Elicitation Techniques: Comparative Study - ResearchGate, https://www.researchgate.net/publication/388156131_Requirements_Elicitation_Techniques_Comparative_Study
4. Requirements elicitation - Wikipedia, https://en.wikipedia.org/wiki/Requirements_elicitation
5. Requirements Elicitation Case Studies Using IBIS, JAD, and ARM - Software Engineering Institute, https://www.sei.cmu.edu/documents/424/2013_019_001_297234.pdf
6. Requirement Elicitation: Advanced Techniques for Accuracy - DhiWise, <https://www.dhiwise.com/blog/requirement-builder/advanced-requirement-elicitation-techniques>
7. Ambiguity and Tacit Knowledge in Requirements Elicitation Interviews - ISTI Open Portal, https://openportal.isti.cnr.it/data/2016/353983/2016_353983.postprint.pdf
8. A Guide to Requirements Elicitation for Product Teams - Jama Software, <https://www.jamasoftware.com/requirements-management-guide/requirements-gathering-and-management-processes/a-guide-to-requirements-elicitation-for-product-teams/>
9. Requirements Elicitation - Software Engineering - GeeksforGeeks, <https://www.geeksforgeeks.org/software-engineering/software-engineering-requirements-elicitation/>
10. Challenges in eliciting requirements - Software Engineering - GeeksforGeeks, <https://www.geeksforgeeks.org/software-engineering/software-engineering-challenges-eliciting-requirements/>
11. How to Handle Conflicting Requirements in Business Systems - Visure Solutions, <https://visuresolutions.com/alm-guide/conflicting-requirements/>
12. Business Analysis: The Elicitation Process - Universal Class, <https://www.universalclass.com/articles/business/project-management/business-analysis-the-elicitation-process.htm>
13. 8 Powerful Methods for Better Requirements Elicitation - SoftKraft, <https://www.softkraft.co/requirements-elicitation/>
14. Requirement Elicitation Techniques - UMSL, https://www.umsl.edu/~sauterv/analysis/6840papers_f12/Chen/index.html
15. A Guide to Requirement Elicitation in Business Analysis, <https://agilemania.com/top-requirement-elicitation-techniques-in-business-analysis>
16. Requirements Elicitation Techniques Part 10 – Prototyping ..., <https://professorpm.com/2024/06/03/requirements-elicitation-techniques-part-10-prototyping-4/>
17. 9.1. When and How to Use Ethnography – The Craft of Sociological Research, <https://pressbooks.rampages.us/sociology-research-methods/chapter/9-1-when-and-how-to-use-ethnography/>
18. Comparison Of Different Techniques For Software Requirement Elicitation, <https://ctts.in/cttsold/assets/upload/4423CTTS100101.pdf>
19. What Questions Do I Ask During Requirements Elicitation? - Bridging the Gap, <https://www.bridging-the-gap.com/what-questions-do-i-ask-during-requirements-elicitation/>
20. What are the differences between throwaway and evolutionary prototypes?, <https://softwareengineering.stackexchange.com/questions/109409/what-are-the-differences-between-throwaway-and-evolutionary-prototypes>
21. Software prototyping - Wikipedia,

https://en.wikipedia.org/wiki/Software_prototyping 22. What is the difference between use case, user requirements and user stories? [closed] - Software Engineering Stack Exchange, <https://softwareengineering.stackexchange.com/questions/235039/what-is-the-difference-between-use-case-user-requirements-and-user-stories> 23. User Stories vs Use Cases - Key Difference & Examples - UXCam, <https://uxcam.com/blog/user-stories-vs-use-cases/> 24. Use Cases and User Stories for Agile Requirements Webinar - YouTube, <https://www.youtube.com/watch?v=NB2ZB6DOrIM> 25. Ethnography: UX Research Methods for Discovery - User Interviews, <https://www.userinterviews.com/ux-research-field-guide-chapter/ethnography> 26. Supporting the System Requirements Elicitation through Collaborative Observations, https://www.researchgate.net/publication/221513279_Supporting_the_System_Requirements_Elicitation_through_Collaborative_Observations 27. 5 Common Challenges in Requirement Elicitation - caw.tech, <https://caw.tech/5-common-challenges-in-requirement-elicitation/>